

WEBPROGRAMOZÁS

IDŐZÍTŐK. ADATTÁROLÁS.
ŰRLAPOK, KÉPEK, TÁBLÁZATOK.
TOVÁBBI NYELVI ELEMEEK

Horváth Győző
egyetemi docens
horvath.gyozo@inf.elte.hu

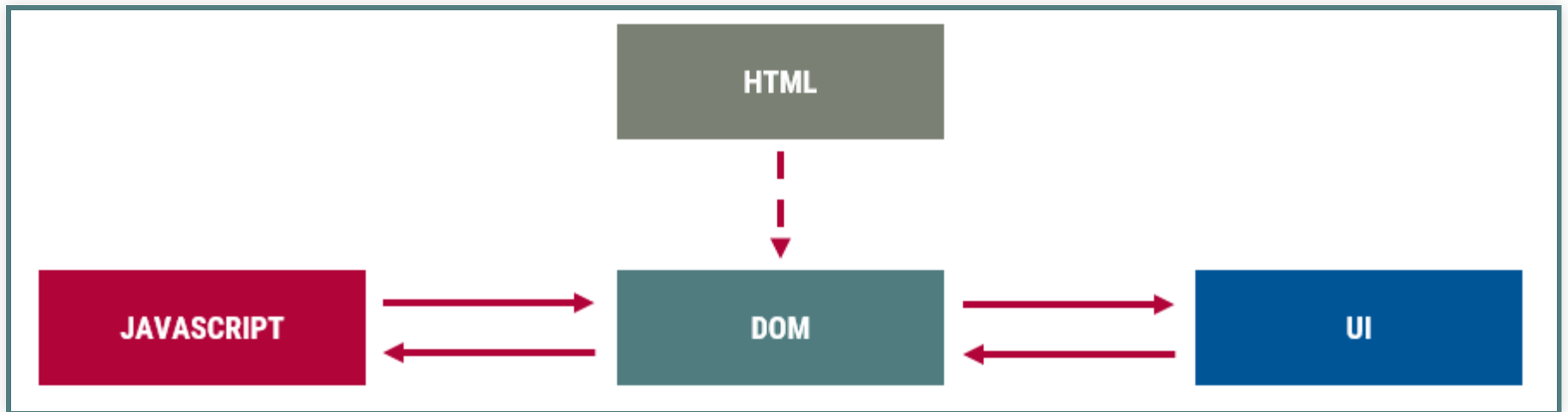
1117 Budapest, Pázmány Péter sétány 1/c., 2.408
Tel: (1) 372-2500/8469



ISMÉTLÉS

ALAPOK

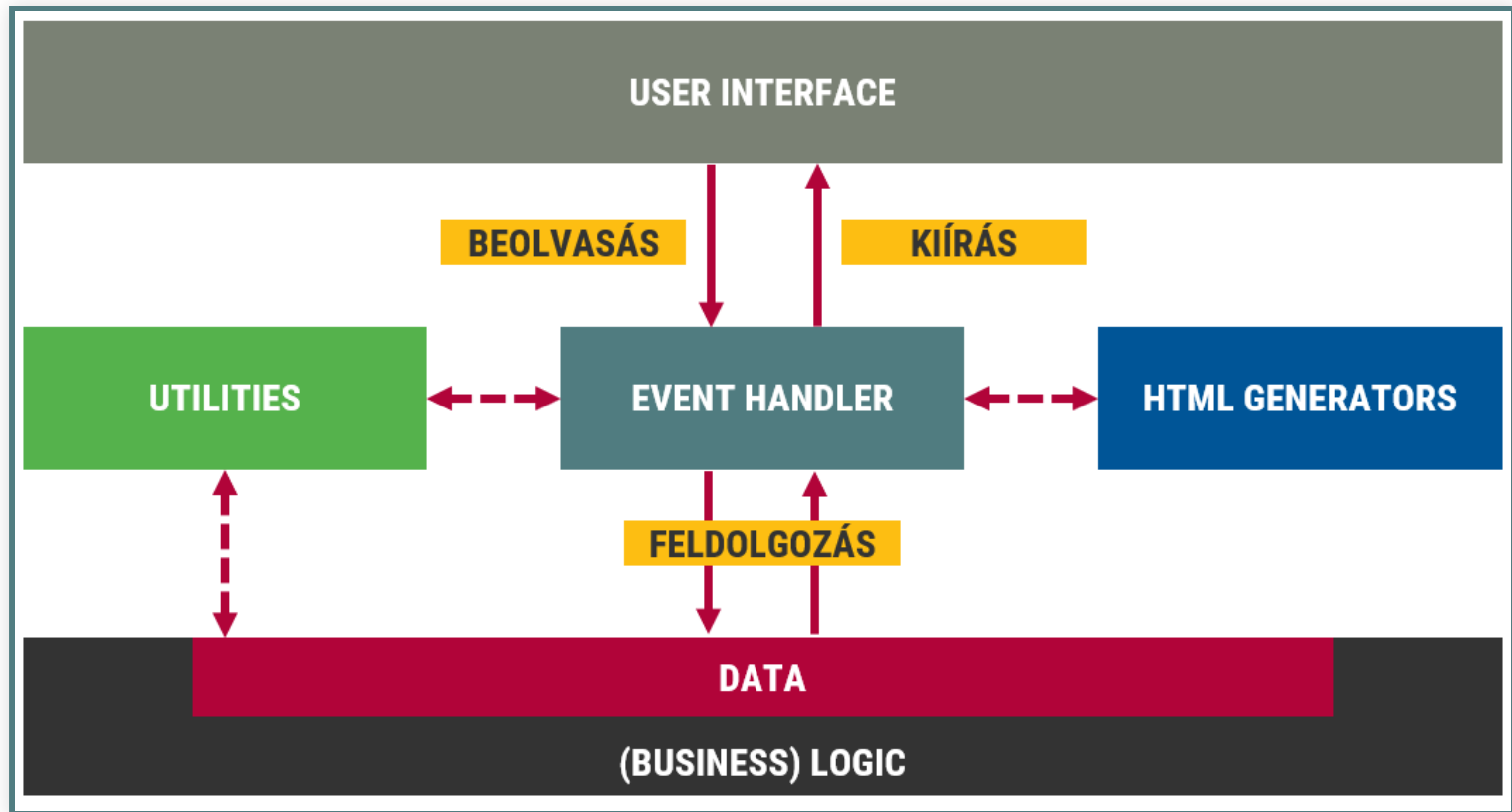
- ✓ JavaScript nyelvi elemei
- ✓ DOM programozás
- ✓ Eseménykezelés részletei



KÓDSZERVEZÉS

- ✓ fizikai, logikai csoportosítás
 - fájl, modul, függvény, osztály
- ✓ egységbe záras
 - objektum, osztály, függvény, modul
- ✓ adattárolás
 - adat és megjelenítés szétválasztása
 - DOM-ban
 - **Nyelvi struktúrában**
- ✓ kiírás a DOM-ba
 - imperatív
 - deklaratív

ALKALMAZÁS RÉSZEI



FELADATMEGOLDÁS LÉPÉSEI

1. Felhasználói felület (UI)

- Tervezés
- HTML, CSS

2. Üzleti logika (BL)

- adatok
- függvények

3. Eseménykezelő függvények

- beolvasás (DOM)
- feldolgozás (BL)
- kiírás (DOM)



IDŐZÍTŐK

setTimeout

- `timerId = setTimeout(fn, ms)`
 - Egy adott függvény futtatása `ms` ms múlva
- `clearTimeout(timerId)`
 - A `timerId`-jú időzítő leállítása

```
// Külön függvény
function tick() {
  console.log('tick');
}
setTimeout(tick, 1000);

// Helyben függvény
setTimeout(function () {
  console.log('tick');
}, 1000);

// Időzítő törlése
const timer = setTimeout(() => {}, 1000);
// do something, or even in an event:
clearTimeout(timer);
```


setInterval

- `timerId = setInterval(fn, ms)`
 - Egy adott függvény futtatása `ms` ms-onként
- `clearInterval(timerId)`
 - A `timerId`-jú időzítő leállítása

```
// Külön függvény
function tick() {
  console.log('tick');
}
setInterval(tick, 1000);

// Helyben függvény
setInterval(function () {
  console.log('tick');
}, 1000);

// Időzítő törlése
const timer = setInterval(() => {}, 1000);
// do something, or even in an event:
clearInterval(timer);
```

IDŐZÍTŐ HASZNÁLATA

- Késleltetett végrehajtás
- Újrarajzolás megvárása (pl. animáció)
- Emberi léptékű műveletvégrehajtás (pl. animáció)
 - hosszú folyamat késleltetése
- Hosszú műveletek felbontása

ÚJRARAJZOLÁS MEGVÁRÁSA

```
<p>Before</p>
<p id="par">Paragraph to fade out/in</p>
<p>After</p>
```

```
const p = document.querySelector('#par');
p.style.transition = 'opacity 1s';

function fadeOut() {
  p.style.opacity = 0;
  p.addEventListener('transitionend', vege, {once:
}

function fadeIn() {
  p.removeEventListener('transitionend', vege)
  p.style.display = '';
  requestAnimationFrame(function () { // waiting
    requestAnimationFrame(function () { // before
      p.style.opacity = 1;
    })
  });
}

function vege() {
```

```
p.style.display = 'none';  
}
```

Before

Paragraph to fade
out/in

After

Fade out

Fade in

HOSSZÚ FOLYAMAT KÉSLELTETÉSE

Ciklustól az időzítőig

```
function vegrehajtas() {  
  for (let i = 0; i < 5; i++) {  
    console.log("feldolgoz", i)  
  }  
}
```

// A számlálós ciklust átírhatjuk előltesztelésre

```
function vegrehajtas(i = 0) {  
  while (i < 5) {  
    console.log("feldolgoz", i)  
    i = i + 1  
  }  
}
```

*// Ebből már könnyű az ismétlődést rekurzióval megoldani,
// tulajdonképpen a `while` helyett `if`-et kell írni,
// és a végén egy rekurzív hívást elhelyezni.*

HOSSZÚ FOLYAMAT FELBONTÁSA

- Ugyanaz, csak azonnal végrehajtva
- Következő iterációba időzíti
- Alternatíva: web worker

```
// Normal, intensive, operation
const table = document.getElementsByTagName("tbody");
for (let i = 0; i < 2000; i++) {
  const tr = document.createElement("tr");
  for (let t = 0; t < 6; t++) {
    const td = document.createElement("td");
    td.appendChild( document.createTextNode(" " + t);
    tr.appendChild( td );
  }
  table.appendChild( tr );
}

// Using a timer to break up a long-running task
const table = document.getElementsByTagName("tbody");
let i = 0;
const max = 1999;
setTimeout(function () {
```


A dramatic landscape photograph featuring a vibrant rainbow arching across a sky filled with dark, textured clouds. The lower portion of the image shows a field of tall, golden-brown grass. In the distance, three silhouetted figures are visible, standing and looking towards the horizon. A semi-transparent dark banner with white text is overlaid across the middle of the image.

ADATMENTÉS A BÖNGÉSZŐBEN

TÁROLÁS

- Változók
- (Süti)
- HTML 5 Storage (Web Storage)
 - `localStorage`
 - `sessionStorage`
- Web SQL Database (részben támogatott)
- Indexed Database (IndexedDB)
- File API (bináris állományok tárolása)

LOCALSTORAGE API

- Adatok tárolása a helyi gépen
- Kulcs-érték párok gyűjteménye
- Csak egyszerű értékek tárolhatók → szerializáció

```
// Storing values
localStorage.setItem("foo") = 42;
// or
localStorage.bar = 42;

// Reading values
console.log(localStorage.getItem("foo"));
// or
console.log(localStorage.bar);

// Storing complex data
const data = [1, 3, 5, 7];
localStorage.setItem('data', JSON.stringify(data));
const loadedData = JSON.parse(localStorage.getItem('data'));
```


A dramatic sunset scene with the Golden Gate Bridge visible through a thick layer of fog. The sky is filled with vibrant orange and red clouds, while the bridge's lights are visible through the mist below.

ŰRLAPOK, KÉPEK ÉS TÁBLÁZATOK

ŰRLAPOK ÉS ŰRLAPELEMEK

- Interakció elsődleges eszközei
- Rengeteg attribútum
- Analóg megközelítés működik
 - `tabindex` → `tabIndex`
 - `maxlength` → `maxLength`
 - `placeholder` → `placeholder`
- Koncentráljunk az ezeken túli tulajdonságokra!

ESEMÉNYEK

- form
 - `submit`, `reset` – megakadályozható
- űrlapelemek
 - `focus`, `blur`
 - `change` (elem elhagyásakor)
 - `input` (`value` változásakor)
 - `invalid`, `search`
- szöveges elemek
 - `keydown`, `beforeinput` – megakadályozható
 - `input`, `keyup`

METÓDUSOK

- form
 - `submit()`, `reset()`
- űrlapelemek
 - `click()`
 - `focus()`, `blur()`
- szöveges elemek
 - `select()`
- `step`-es elemek
 - `stepDown()`, `stepUp()`

TULAJDONSÁGOK

■ form

- `elements` (`HTMLFormControlsCollection`)
 - `elements['id_or_name']` (Element vagy `RadioNodeList`)

■ űrlapelemek

- `form`, `formAction`, `formEncType`, stb.
- `defaultValue`, `defaultChecked`
- `defaultSelected` (`<option>` elem)
- `valueAsDate`, `valueAsNumber`
- `indeterminate` (checkbox, radio)

RADIO

```
<form action="">
  <input type="radio" name="fruit" id="fruit1" value="apple">
  <input type="radio" name="fruit" id="fruit2" value="pear"><
  <input type="radio" name="fruit" id="fruit3" value="plum"><
  <input type="radio" name="fruit" id="fruit4" value="lemon">
</form>
```

```
document.addEventListener('click', function (e) {
  if (e.target.matches('[type=radio]')) {
    // RadioNodeList.value
    const value = document.querySelector('form').elements['fruit'];
    console.log(value);
  }
})
```

☐ Apple ☐ Pear ☐ Plum ☐ Lemon

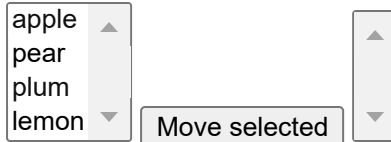
SELECT

■ Select

- value
- selectedIndex
- selectedOptions
- options
- add(option[, before]),
remove(index)

■ Option

- index
- text



apple
pear
plum
lemon

Move selected

SELECT

```
const from = document.querySelector('select[name=from]')
const to = document.querySelector('select[name=to]')
document.querySelector('button').addEventListener('click', function() {
  const selectedOptions = Array.from(from.selectedOptions)
  selectedOptions.forEach(o => to.add(o))
})
```

TEXT SELECTION API

- űrlapelemek
 - text/password/search/tel/url/week/month
- Tulajdonságok
 - `selectionStart`, `selectionEnd`, `selectionDirection`
- Metódusok
 - `setSelectionRange()`, `setRangeText()`

```
<input type="text" value="example text">  
<button id="btn1">log selection</button>  
<button id="btn2">set selection (2, 5)</button>
```

```
const input = document.querySelector('input')  
document.querySelector('#btn1').addEventListener('click', function()  
  console.log(input.selectionStart, input.selectionEnd)  
})  
document.querySelector('#btn2').addEventListener('click', function()  
  input.setSelectionRange(2, 5)  
})
```

ŰRLAPELLENŐRZÉS – HTML ÉS CSS

■ HTML5 attribútumok

- novalidate
- required
- pattern
- minlength, maxlength
- min, max, step

■ CSS pseudo-szelektorok

- :valid
- :invalid
- :in-range
- :out-of-range
- :required
- :optional
- :read-only
- :read-write
- :checked

ŰRLAPELLENŐRZÉS – JAVASCRIPT

HTML5 constraint validation API

- `invalid` esemény
- `validationMessage`: hibaüzenet
- `willValidate`: lesz-e ellenőrizve
- `validity`: `ValidityState`
 - `patternMismatch`, `valueMissing`, `tooLong`, `valid`, ...
- `checkValidity()`: logikai (`invalid` esemény)
- `reportValidity()`: logikai (form elem)
- `setCustomValidity(üzenet)`: egyedi hibaüzenet

Példa és leírás

ŰRLAPELLENŐRZÉS 1.

Egyszerű űrlap

```
<form action="">  
  <button type="submit" name="btn">Submit</button>  
</form>
```

A screenshot of a web browser displaying a simple form. The form is contained within a light blue rectangular frame. In the top-left corner of the form area, there is a single button with the text 'Submit' on it. The rest of the form area is empty.

ÚRLAPELLENŐRZÉS 2.

submit esemény, küldés megakadályozása

```
document.querySelector('form').addEventListener('submit', function(e) {  
  console.log(e);  
  e.preventDefault();  
})
```

Submit

ÚRLAPELLENŐRZÉS 3.

HTML validátorok használata

```
<form action="">
  <p>Kötelező szöveg: <input type="text" required><span></span>
  <p>RegExp szöveg: <input type="text" pattern="[A-Z]{3}-\d{3}"
  <p>Minlength, maxlength: <input type="text" minlength="3" max
  <p>Szám mező: <input type="number" value="1" min="1" max="10"
  <p>Email mező: <input type="email" required><span></span></p>
  <button type="submit" name="btn">Submit</button>
</form>
```

Kötelező szöveg:

RegExp szöveg:

Minlength, maxlength:

Szám mező:

Email mező:

ÚRLAPELLENŐRZÉS 4.

HTML validátorok + CSS

```
input:valid {  
    box-shadow: 0 0 5px 1px green;  
}  
input:invalid {  
    box-shadow: 0 0 5px 1px red;  
}  
input:focus {  
    box-shadow: none;  
}  
  
input:valid + span:after {  
    content: " ✓";  
    color: green;  
}  
input:invalid + span:after {  
    content: " ✗";  
    color: red;  
}
```

Kötelező szöveg: ✗

RegExp szöveg: ✗

Minlength, maxlength: ✓

Szám mező: ✓

Email mező: ✗

ÚRLAPELLENŐRZÉS 5.

Saját hibaüzenet

```
const input = document.querySelector('input')
input.addEventListener('input', function (e) {
  if (this.validity.patternMismatch) {
    this.setCustomValidity("Bad text according to the pattern (")
  } else {
    this.setCustomValidity("")
  }
  this.nextElementSibling.innerHTML = this.validationMessage
})
```

RegExp szöveg: ✖

Submit

ÚRLAPELLENŐRZÉS 6.

Saját hibaüzenet invalid eseménnyel

```
const input = document.querySelector('input')
input.addEventListener('input', function (e) {
  this.setCustomValidity("")
  this.checkValidity();
  this.nextElementSibling.innerHTML = this.validationMessage
})
input.addEventListener('invalid', function (e) {
  if (this.validity.patternMismatch) {
    this.setCustomValidity("Bad text according to the pattern (
  } else {
    this.setCustomValidity("")
  }
  this.nextElementSibling.innerHTML = this.validationMessage
})
```

RegExp szöveg: ✖

Submit

ŰRLAPELLENŐRZÉS 7.

novalidate

```
<form action="" novalidate>
  <p>RegExp szöveg: <input type="text" pattern="[A-Z]{3}-\d{3}"
  <button type="submit" name="btn">Submit</button>
</form>
```

RegExp szöveg: ✖

Submit

ÚRLAPELLENŐRZÉS 8.

Saját validáció

```
<form action="" novalidate>
  <p>From: <input type="time" name="from" required
  <p>To: <input type="time" name="to" required val
  <button type="submit" name="btn">Submit</button>
</form>
```

```
const form = document.querySelector('form')
const from_field = document.querySelector("[name=f
const to_field = document.querySelector("[name=to]

const padTime = timeString => timeString.split(":")
function checkTime() {
  const from = padTime(from_field.value)
  const to = padTime(to_field.value)
  return from <= to
}
form.addEventListener('submit', function (e) {
  if (checkTime()) {
    from_field.setCustomValidity("")
  } else {
```



```

        from_field.setCustomValidity("from < to")
    }

    if (!this.reportValidity()) {
        e.preventDefault()
    }
})
form.addEventListener('input', function (e) {
    if (e.target.matches('input[type=time]')) {
        if (checkTime()) {
            from_field.setCustomValidity("")
        } else {
            from_field.setCustomValidity("from < to")
        }
    }
})

```

From: ✓

To: ✓

KÉPEK

- ``
- Legfontosabb tulajdonsága: `src`
 - Dinamikusan lecserélni a képet
- Tipikus képműveletek
 - Kép lecserélése
 - Kép előtöltése

```
const mem_kep = document.createElement('img');  
mem_kep.src = 'korte.png';  
//...  
//ha szükséges a csere:  
const kep = document.querySelector('img');  
kep.src = mem_kep.src;
```

KÉP ELŐTÖLTÉSE ÉS CSERÉJE

```
<img src="">
```

```
const images = [
  'https://source.unsplash.com/NRQV-hBF10M/640x480',
  'https://source.unsplash.com/VJTmFSendQ0/640x480',
  'https://source.unsplash.com/1q5KmcSe760/640x480',
  'https://source.unsplash.com/fBRVaRdFMIw/640x480',
  'https://source.unsplash.com/CyhYe8B9PuY/640x480',
  'https://source.unsplash.com/45bI5ezo3gE/640x480',
]
let index = 0
const img = document.querySelector('img')
const prevImage = document.createElement('img')
const nextImage = document.createElement('img')
function nextIndex(i) {
  return (i + 1) % images.length
}
function prevIndex(i) {
  return i === 0 ? images.length - 1 : i - 1
```

KÉP ELŐTÖLTÉSE ÉS CSERÉJE



TÁBLÁZAT

■ Táblázat

- rows
- insertRow(index)
- deleteRow(index)

■ Sor

- rowIndex
- sectionRowIndex
- cells
- insertCell(index)
- deleteCell(index)

■ Cella

- cellIndex

```
function xyKoord(td) {  
  const x = td.cellIndex  
  const tr = td.parentNode  
  const y = tr.sectionRowIndex  
  return {x, y}  
}
```

TÁBLÁZAT – PÉLDA

```
const table = document.querySelector('table')
table.addEventListener('click', function (e) {
  if (e.target.matches('td')) {
    const {x, y} = xyKoord(e.target)
    table.rows[y].cells[x].classList.toggle('piros')
    table.querySelector('caption').innerHTML = `${x}, ${y}`
  }
})
```


TOVÁBBI NYELVI ELEMEEK

`typeof` OPERÁTOR

Adott érték típusa szövegesen

```
typeof undefined    === "undefined"
typeof null         === "object"
typeof true         === 'boolean'
typeof 37           === 'number'
typeof 42n          === 'bigint'
typeof NaN          === 'number'
typeof 'bla'        === 'string'
typeof {a: 1}       === 'object'
typeof [1, 2, 4]    === 'object' // Array.isArray()
typeof new Date()   === 'object'
typeof function() {} === 'function'
```

Referencia

NUMBER

■ Konstans

- `Number.NaN` (`NaN`)
- `Number.POSITIVE_INFINITY` (`Infinity`)
- `Number.NEGATIVE_INFINITY` (`-Infinity`)

■ Statikus metódusok

- `Number.isNaN(value)`
- `Number.isFinite(value)`
- `Number.isInteger(value)`
- `Number.parseInt(str[, radix])`
- `Number.parseFloat(str)`

Referencia

NUMBER

```
// NaN
const a = 'alma' / 2           // NaN
Number.isNaN(a)                // true
3 + Number.NaN                 // NaN

// Infinity
const a = 10/0
isFinite(a)                     //false

// parseInt, parseFloat
Number.parseInt('123')          //123
Number.parseInt('123', 10)      //123
Number.parseInt('0101', 2)      //5
Number.parseInt('alma', 10)     //NaN
Number.parseInt('5alma', 10)    //5
Number.parseFloat('4.54')       //4.54
```

NUMBER

■ Példánymetódusok

- `numObj.toExponential([fractionDigits])`
- `numObj.toFixed([digits])`: tizedespont utáni számjegyek száma
- `numObj.toPrecision([precision])`: számjegyek száma
- `numObj.toString([radix])`

```
// boxing: literal to numObj
3.toString()           // error
(3).toString()         // "3"
Number.parseInt(3).toString() // "3"
const n = 3
n.toString()           // "3"

// példánymetódusok
(123.456).toExponential(2) // "1.23e+2"
(123.456).toFixed(2)       // "123.46"
(123.456).toPrecision(2)   // "1.2e+2"
(123.456).toPrecision(4)   // "123.5"
(123).toString(2)          // "1111011"
```

BIGINT

$2^{53}-1$ -nél nagyobb egész számok ábrázolására

Operátorok: +, -, *, /, **

```
const bigint1 = 42n
const bigint2 = BigInt(42)
const bigint3 = BigInt("42")
const bigint4 = BigInt(Number.MAX_SAFE_INTEGER) // 900719925474
const bigint5 = bigint4 + 1n // 900719925474

// https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/BigInt
function isPrime(p) {
  for (let i = 2n; i * i <= p; i++) {
    if (p % i === 0n) return false;
  }
  return true;
}
```

DATE

- `Date` objektum
- Aktuális időpont
 - `Date.now()`
- Megadott időpont
 - `new Date(year, month, day [, hour, minute, second, millisecond]);`
 - `Date.parse(dateString): ms`
- Műveletcsoportok
 - getterek: `getMonth()`, `getDay()`, stb
 - setterek: `setMonth()`, `setDay()`, stb.
 - `toString` metódusok: pl. `toLocaleString()`, `toGMTString()`

Referencia

DATE PÉLDA

```
// Létrehozás
const d1 = new Date()
const d2 = new Date(2019, 0, 31, 3, 42, 0);
const d3 = new Date("2019-01-31T03:42:00");
const ms1 = Date.now();
const ms2 = Date.parse("2019-01-31T03:42:00");

// Műveletek
d2.getFullYear()           // 2019
d3.setFullYear(2020)
d2.toLocaleString('hu-HU') // "2019. 01. 31. 3:42:00"
d2.toLocaleString('en-US') // "1/31/2019, 3:42:00 AM"

// Eltelt idő (ms-okban)
const delta = ms2 - ms1
```

STRING MŰVELETEK

- egy karakter vagy kódja
 - `charAt(i)`, `charCodeAt(i)`
- keresés előlről, hátulról, indexet ad vissza
 - `indexOf(mit, honnan)`, `lastIndexOf(mit, honnan)`
- összehasonlítás, -1, 0, 1
 - `localeCompare(mivel[, locale])`
- keresés, csere, reguláris kifejezésekkel is
 - `search(regex)`, `replace(mit, mivel)`, `match(regex)`,
`matchAll(regex)`
- részszoveg
 - `substr(honnan, hossz)`, `substring(mettől, meddig)`,
`slice(mettől, meddig)`

STRING MŰVELETEK

■ szöveg felbontása → tömb

- `split(szeptátor)`

■ kezdődik, végződik

- `startsWith(str)`, `endsWith(str)`

■ kitöltés

- `padStart(length[, str])`, `padEnd(length[, str])`

■ ismételt

- `repeat(num)`

■ fehérközök eltávolítás

- `trim()`, `trimStart()`, `trimEnd()`

■ kisbetű, nagybetű

- `toLowerCase()`, `toUpperCase()`,
`toLocaleLowerCase(locale)`, `toLocaleUpperCase(locale)`

STRING PÉLDA

```
'piros alma'.charAt(2)           // "r"
'piros alma'.charCodeAt(2)       // 114
'piros alma'.indexOf('alma')     // 6
'piros alma'.localeCompare('piros körte') // -1
'piros alma'.replace('piros', 'sárga') // "sárga alma"
'piros alma'.substr(2, 3)        // "ros"
'piros alma'.split(' ')         // ["piros","alma"]
'piros alma'.startsWith('piros') // true
'piros alma'.padStart(15)       // "      piros alma"
'      piros alma'.trim()       // "piros alma"
'piros alma'.toUpperCase()     // "PIROS ALMA"
```

SZÖVEGKÓDOLÁS

- Speciális karakterek
 - tárolása
 - küldése
- Teljes URL kódolása
 - `encodeURIComponent()`, `decodeURI()`
- URL-beli érték kódolása
 - `encodeURIComponent()`, `decodeURIComponent()`

```
const kod = encodeURIComponent('árvíztűrőtükörfúrógép');  
//"%C3%A1rv%C3%ADzt%C5%B1r%C5%91t%C3%BCk%C3%B6rf%C3%BAr%C3%B3g%C3%A9p"  
const dekod = decodeURIComponent(kod);  
//"árvíztűrőtükörfúrógép"  
const url = encodeURI('http://server.hu/útvonal szóközökkel');  
//"http://server.hu/%C3%BAtvonal%20sz%C3%B3k%C3%B6z%C3%B6kkel"
```

REGEXP

- Reguláris kifejezés
- Olyan minta, amely szövegekre illeszthető
- Literálforma: `/minta/flags`
 - `minta`: normál szöveg + speciális jelölők
 - `flag`: viselkedést szabályozó jelölők
- Metódusok
 - `test(str)` → logika érték
 - `exec(str)` → tömb
- String függvények használják intenzíven
 - `search(regex)`
 - `replace(regex, mire)`
 - `match(regex)`, `matchAll(regex)`
 - `split(regex)`

REGEXP PÉLDA

```
/\d+/.test('alma')      //false
/\d+/.test('alma123')    //true
/alma/.test('piros alma kukacos'); // true
/alma$/.test('piros alma kukacos') //false
/^\w+/.test('piros alma') //true
/alma/i.test('Piros Alma') //true
"piros alma kukacos".replace(/os/, 'itott') // "píritott alma k
"piros alma kukacos".replace(/os/g, 'itott') // "píritott alma k

"1a234 5b678 90"
  .replace(/\D/g, '') // "1234567890"
  .replace(/.{4}/g, '$& ') // "1234 5678 90"

"Horvath Gyozo".replace(/(\w+)\s(\w+)/i, "$2, $1") // "Gyozo, H
'Hófehérke és a 7 törpe meg az 1 herceg'.match(/\d+/g) // ["7",
/((\w+)\s(\w+))/ig.exec("Horvath Gyozo Horvath Emese")
```

MATH

■ Matematikai műveletek

- `sin(rad)`, `asin(sz)`, `cos(rad)`, `acos(sz)`, `tan(rad)`,
`atan(sz)`
- `pow(alap, kit)`, `exp(n)`, `log(n)`, stb.
- `random()`
- `round(n)`, `floor(n)`, `ceil()`

■ Konstansok

- `PI`
- `E`
- `SQRT2`
- `LN2`, stb.

MATH PÉLDA

```
Math.PI //3.141592653589793
Math.sin(90) //0.8939966636005579
Math.sin(90 * Math.PI / 180) //1
Math.random() //p1. 0.47057095554085615
Math.random() //p1. 0.5286946792885748
Math.round(1.6) //2
Math.floor(1.6) //1
```

ARRAY

- `pop()`, `push(e)`, `shift()`, `unshift(e)`
 - végéről, végére, elejéről, elejére
- `reverse()`
 - megfordít
- `splice(honnan, mennyit)`
 - kivág
- `join(szeparátor)`
 - szöveggé fűz
- `indexOf(elem)`
 - keresés
- `includes(elem)`
 - eldöntés

ARRAY PÉLDA

```
const t = [1, 2, 3, 4, 5]
t.push(6)           // [1, 2, 3, 4, 5, 6]
t.pop()             // 6, [1, 2, 3, 4, 5]
t.unshift(0)        // [0, 1, 2, 3, 4, 5]
t.shift()           // 0, [1, 2, 3, 4, 5]
t.reverse()         // [5, 4, 3, 2, 1]
t.sort()            // [1, 2, 3, 4, 5]
t.splice(2, 1)       // [5, 4, 2, 1]
t.join('###')        // "5###4###2###1"

[1, 2].concat([3, 4])           // [1, 2, 3, 4]
[1, 2, 3, 4, 5].copyWithin(2, 0, 2) // [1, 2, 1, 2, 5]
[1, 2, 3].fill(4)               // [4, 4, 4]
[1, 2, [3, 4]].flat()           // [1, 2, 3, 4]
[1, 2, 3, 4].flatMap(x => [x * 2]) // [2, 4, 6, 8]
[1, 2, 3, 4, 5].slice(2, 4)     // [ 3, 4 ]
```


MAP

- `set(key, value)`
- `get(key)`
- `has(key)`
- `delete(key)`
- `size`
- `keys()`, `values()`,
`entries()`
- `clear()`

```
const map = new Map()
map.set('some', 'value')
map.get('some')           // 'value'
map.has('some')           // true
map.delete('some')

const map2 = new Map([['first', 1], [
  for (const [key, value] of map2.entries()
    console.log(key + ' = ' + value);
  ]])
// 'first' = 1
// 'second' = 2
```

SET

- `add(value)`
- `has(value)`
- `delete(value)`
- `size`
- `values()`
- `clear()`

```
const set = new Set()
set.add(1)
set.has(1)           // true
set.delete(1)

const set2 = new Set([1, 2, 3, 4, 1,
for (const value of set2) {
    console.log(value);
}
// 1, 2, 3, 4

// https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global\_Objects/Set
function isSuperset(set, subset) {
    for (var elem of subset) {
        if (!set.has(elem)) {
            return false;
        }
    }
    return true;
}
```

ÖSSZEFOGLALÁS

- Időzítők és példák
- Adattárolás a `localStorage` API-val
- Űrlapok, képek és táblázatok
- További nyelvi elemek
 - JavaScript beépített objektumai

